TryFit - Virtual Try-On Fashion App

Complete Documentation

Table of Contents

1. Overview
2. User Guide
    - Quick Start
    - Features Walkthrough
    - How-To Guides
    - FAQ & Troubleshooting
3. Technical Documentation
    - System Architecture
    - Database Schema
    - API Reference
    - Frontend Structure
    - Computer Vision Integration
    - Authentication Flow
4. Deployment Guide
    - Environment Variables
    - Database Setup
    - Running Locally
    - Publishing to Replit
    - Admin Configuration
    - Production Considerations

Overview

TryFit is a web-based virtual try-on application that allows users to visualize how clothing items would look on them without physically trying them on. The app uses computer vision (MediaPipe Pose) to detect body landmarks and overlay clothing items onto user photos or pre-selected model photos.

Core Features

- **Virtual Try-On**: Upload your photo and see how clothes look on you
- **Body Pose Detection**: AI-powered detection of 33 body landmarks for accurate clothing placement
- **Multi-Layer Outfits**: Try on multiple items at once (top + bottom + accessories)
- **Size Recommendations**: Get personalized size suggestions based on your body measurements
- **Model Gallery**: Use pre-photographed models if you prefer not to upload your own photo
- **Personal Closet**: Save favorite items for quick access
- **Social Sharing**: Share your try-on results and get feedback from friends
- **Admin Dashboard**: Manage the clothing catalog and models (for administrators)

Technology Stack

Layer	Technology
Frontend	React, TypeScript, Vite, Tailwind CSS, Shadcn UI
Backend	Node.js, Express.js
Database	PostgreSQL (Neon serverless)
ORM	Drizzle ORM
Authentication	Replit Auth (OpenID Connect)
Computer Vision	MediaPipe Pose

Layer	Technology
State Management	Zustand, TanStack Query

User Guide

Quick Start

Getting Started in 5 Minutes

1. **Sign In**: Click the "Login with Replit" button to create your account
2. **Upload a Photo**: Go to the Upload page and either:
    - Drag and drop a full-body photo
    - Use your camera to take a new photo
    - Select a pre-photographed model
3. **Wait for Body Detection**: The app will automatically detect your body pose
4. **Browse Clothes**: Navigate to the Catalog and select items you want to try on
5. **Try It On**: Click "Try On" to see the clothing overlaid on your photo
6. **Adjust & Save**: Fine-tune the fit and save your favorite looks

Best Photo Tips

For the best virtual try-on experience:

- Use a **full-body photo** (head to toe visible)
- Stand in a **neutral pose** with arms slightly away from your body
- Use **good lighting** without harsh shadows
- Wear **form-fitting clothes** in your reference photo
- Use a **plain background** for best detection accuracy

Features Walkthrough

Home Page

The home page shows:

- **Recent Try-Ons**: Your latest virtual try-on sessions
- **Featured Items**: Popular clothing items to explore
- **Quick Actions**: Fast access to upload, catalog, and closet

Upload Page

Upload your photo for virtual try-on:

- **Drag & Drop**: Simply drag a photo onto the upload area
- **File Browser**: Click to select a file from your device
- **Camera Capture**: Use your device camera (on supported devices)
- **Pose Detection**: Automatic body landmark detection runs after upload

Catalog Page

Browse the clothing collection:

- **Categories**: Filter by Tops, Bottoms, Dresses, Outerwear, Accessories
- **Browse**: View available items in each category
- **Quick Actions**: Add to closet or try on directly

Studio Page

The main virtual try-on workspace:

- **Canvas View**: See your photo with clothing overlays
- **Item Controls**: Adjust position, scale, and rotation of clothing
- **Layer Management**: Reorder items for proper layering
- **Shadow & Lighting**: Adjust shadows and brightness for realistic blending
- **Size Recommendation**: View suggested sizes based on your measurements
- **Save & Share**: Save results and share with friends

Closet Page

Your personal clothing collection:

- **Saved Items**: All items you've favorited

- **Quick Try-On**: One-click to try on saved items
- **Remove Items**: Manage your collection

Models Page

Browse pre-photographed models:

- **Filter by Body Type**: Petite, Average, Athletic, Curvy, Plus Size
- **Filter by Height**: Find models similar to your height
- **Filter by Skin Tone**: Select models with similar complexions
- **Quick Select**: Use any model for your try-on session

Profile Page

Manage your account:

- **Profile Info**: View your account details
- **Try-On History**: See all your past sessions
- **Saved Outfits**: Access your outfit combinations
- **Logout**: Sign out of your account

Shared Results Page

When you share a try-on result:

- **View Link**: Anyone with the link can see your outfit
- **Voting**: Friends can vote (Love, Like, Meh, Not for me)
- **Comments**: Get written feedback on your outfit
- **No Login Required**: Voting and commenting is open to anyone

---

How-To Guides

How to Upload a Photo and Detect Body Pose

1. Navigate to the **Upload** page from the navigation menu
2. Choose your upload method:
    - **Drag & Drop**: Drag your photo onto the upload area

    - **Click to Browse**: Click the upload area and select a file
    - **Camera**: Click the camera icon to capture a new photo
3. Wait for the blue progress indicator while pose detection runs
4. Once complete, you'll see body landmarks overlaid on your photo
5. Click **Continue to Try-On** to proceed to the Studio

How to Try On Clothing

1. From the Catalog, click **Try On** on any clothing item
2. Or from the Studio, click **Add Item** to browse the catalog
3. The clothing will appear on your photo, positioned at body landmarks
4. Use the controls to adjust:
    - **Position**: Drag the item to reposition
    - **Scale**: Use the slider to resize
    - **Rotation**: Rotate for better alignment
5. For multi-layer outfits, add more items (they stack automatically)

How to Get Size Recommendations

1. In the Studio, select a clothing item
2. Look for the **Size Recommendation** panel
3. The app calculates your measurements from body landmarks:
    - Shoulder width
    - Torso height
    - Hip width
4. It compares these to the item's size chart
5. You'll see:
    - **Recommended Size** with confidence percentage
    - **Fit Description** (e.g., "Standard fit", "May be snug")
    - **Alternative Sizes** if the match isn't perfect

How to Save and Share Your Look

1. In the Studio, create your outfit combination
2. Click **Save Result** to store the try-on session
3. To share:
    - Click **Share** to generate a unique link
    - Copy the link and send to friends
    - Friends can view, vote, and comment without logging in
4. View feedback on the shared results page

How to Manage Your Closet

1. **Add to Closet**: On any clothing item, click the heart/save icon
2. **View Closet**: Navigate to the Closet page
3. **Remove Items**: Click the remove icon on any saved item
4. **Quick Try-On**: Click any closet item to try it on immediately

How to Use Pre-Photographed Models

1. Navigate to the **Models** page
2. Use filters to find a model similar to you:
    - Body Type: Petite, Average, Athletic, Curvy, Plus Size (server-side filter)
    - Height: Various height ranges (client-side filter)
    - Skin Tone: Various options (client-side filter)
3. Click on a model to select them
4. Their photo and body landmarks will be used for try-on
5. Proceed to try on clothes as normal

FAQ & Troubleshooting

Frequently Asked Questions

Q: Is my photo stored on your servers? A: Your photos are processed locally in your browser for pose detection. Only saved try-on results are stored on our servers, and these are linked to your account.

Q: Why isn't the pose detection working on my photo? A: Pose detection works best with:

- Full-body photos (head to toe)
- Clear visibility of shoulders, hips, and limbs
- Good lighting without harsh shadows
- Front-facing poses

Q: Can I try on multiple items at once? A: Yes! TryFit supports multi-layer outfits. You can combine tops, bottoms, and accessories in a single try-on session.

Q: How accurate are the size recommendations? A: Size recommendations are based on body measurements calculated from your photo. Accuracy depends on photo quality and pose. Use them as a guide, but always check the specific brand's size chart.

Q: Can friends see my photo when I share a result? A: Yes, the shared link includes the try-on result image which shows the clothing on your photo. Only share links with people you trust.

Q: Do I need an account to vote on shared outfits? A: No, voting and commenting on shared outfits is open to anyone with the link.

Troubleshooting

Issue: Pose detection fails or is inaccurate

- Solution: Try a different photo with better lighting and a clearer pose
- Ensure your full body is visible in the frame
- Use a plain background if possible

Issue: Clothing appears in the wrong position

- Solution: Use the position and scale controls in the Studio to adjust
- Check that body landmarks were detected correctly
- Try re-uploading your photo with a clearer pose

Issue: Cannot log in

- Solution: Ensure you have a Replit account
- Try refreshing the page and clicking login again
- Check that cookies are enabled in your browser

Issue: Images not loading

- Solution: Check your internet connection
- Try refreshing the page
- Clear your browser cache

Technical Documentation

System Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    CLIENT                                │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────────────┐ │
│ │ React        │ │ Zustand      │ │ TanStack Query       │ │
│ │ + Vite       │ │ Store        │ │ (Server State)       │ │
│ └──────────────┘ └──────────────┘ └──────────────────────┘ │
│ ┌──────────────────────────────────────────────────────┐│
│ │        MediaPipe Pose (Browser-side CV)              │
│ └──────────────────────────────────────────────────────┘│
└──────────────────────────────────────────────────────────┘
                  │
                  │ HTTP/REST API
                  ▼
┌──────────────────────────────────────────────────────────┐
│                    SERVER                                │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────────────┐ │
```

```
│ │ Express.js │ │ Passport  │ │ Drizzle ORM       │ │
│ │ Routes    │ │ (OIDC)    │ │ (Type-safe queries) │ │
│ └──────────┘ └──────────┘ └──────────────────────┘ │
└──────────────────────────────────────────────────────┘
                  │
                  │ SQL (WebSocket)
                  ▼
┌──────────────────────────────────────────────────────┐
│            PostgreSQL (Neon)                          │
│ ┌──────────────────────────────────────────────────┐ │
│ │ users │ clothing_items │ try_on_sessions │ models │ ... │  │
│ └──────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────┘
```

Key Components

Component	Purpose	Location
Express Server	API routes, session management	server/
React Frontend	User interface, interactions	client/src/
Drizzle ORM	Database queries, schema	shared/schema.ts, server/db.ts
Zustand Store	Client-side state (photos, landmarks, results)	client/src/store/appStore.ts
MediaPipe Pose	Body landmark detection (runs in browser)	client/src/hooks/usePoseDetec
TanStack Query	Server state caching, data fetching	Throughout client/src/

Database Schema

Entity Relationship Diagram

```
┌──────────────┐   ┌────────────────────┐   ┌────────────────────┐
│  users   │────<│  user_closet   │>────│ clothing_items  │
└──────────────┘   └────────────────────┘   └────────────────────┘

   │
   │      ┌────────────────────┐
   └──────────────<│  saved_outfits   │
   │      └────────────────────┘
   │
   │      ┌────────────────────┐   ┌────────────────────┐
   └──────────────<│ try_on_sessions  │────<│  outfit_votes   │
          │         │   └────────────────────┘
          │         │   ┌────────────────────┐
          │         │────<│ outfit_comments │
          └────────────────────┘   └────────────────────┘


┌──────────────┐   ┌────────────────────┐
│  models   │   │  sessions     │
└──────────────┘   └────────────────────┘
```

Table Definitions

users

Stores user account information.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
email	varchar (unique)	User email address
firstName	varchar	First name
lastName	varchar	Last name
profileImageUrl	varchar	Profile picture URL
bodyProfilePhotoUrl	varchar	Saved body photo URL
bodyMeasurements	jsonb	Calculated body measurements
createdAt	timestamp	Account creation date
updatedAt	timestamp	Last update date

clothing_items

Product catalog of clothing items.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
name	varchar(255)	Item name
brand	varchar(100)	Brand name
category	enum	tops, bottoms, dresses, outerwear, accessories

Column	Type	Description
description	text	Item description
imageUrl	varchar	Main product image
thumbnailUrl	varchar	Thumbnail image
price	decimal(10,2)	Price in dollars
sizesAvailable	text[]	Array of available sizes
sizeChart	jsonb	Size chart measurements
externalPurchaseUrl	varchar	Link to buy
overlayConfig	jsonb	Positioning config for try-on
isActive	boolean	Soft delete flag
createdAt	timestamp	Creation date

user_closet

Many-to-many relationship for saved/favorited items.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
userId	varchar (FK)	References users.id
clothingItemId	varchar (FK)	References clothing_items.id

Column	Type	Description
addedAt	timestamp	When item was saved

saved_outfits

User-created outfit combinations.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
userId	varchar (FK)	References users.id
name	varchar(255)	Outfit name
itemIds	text[]	Array of clothing item IDs
previewImageUrl	varchar	Outfit preview image
createdAt	timestamp	Creation date

try_on_sessions

Stored virtual try-on results.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
userId	varchar (FK)	References users.id (nullable)
userPhotoUrl	varchar	User's photo URL
bodyLandmarks	jsonb	33 MediaPipe pose landmarks

Column	Type	Description
results	jsonb	Try-on result data
shareToken	varchar (unique)	Token for public sharing
createdAt	timestamp	Creation date

models

Pre-photographed models for try-on.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
name	varchar(100)	Model name
photoUrl	varchar	Full body photo
thumbnailUrl	varchar	Thumbnail image
height	varchar(20)	Model height
bodyType	enum	petite, average, athletic, curvy, plus_size
skinTone	varchar(50)	Skin tone description
bodyLandmarks	jsonb	Pre-detected pose landmarks
isActive	boolean	Soft delete flag
createdAt	timestamp	Creation date

outfit_votes

Votes on shared try-on results.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
sessionId	varchar (FK)	References try_on_sessions.id
voterId	varchar (FK)	References users.id (nullable)
voterName	varchar(100)	Name of voter
voteType	varchar(20)	love, like, meh, dislike
createdAt	timestamp	Vote date

outfit_comments

Comments on shared try-on results.

Column	Type	Description
id	varchar (PK)	UUID, auto-generated
sessionId	varchar (FK)	References try_on_sessions.id
authorId	varchar (FK)	References users.id (nullable)
authorName	varchar(100)	Comment author name
content	text	Comment content
createdAt	timestamp	Comment date

sessions

Express session storage (required for authentication).

Column	Type	Description
sid	varchar (PK)	Session ID
sess	jsonb	Session data
expire	timestamp	Expiration time

Enums

// Clothing categories

clothing_category: "tops" | "bottoms" | "dresses" | "outerwear" | "accessories"

// Available sizes

clothing_size: "XS" | "S" | "M" | "L" | "XL" | "XXL"

// Model body types

model_body_type: "petite" | "average" | "athletic" | "curvy" | "plus_size"

API Reference

Authentication

All authenticated endpoints require a valid session cookie. Use Replit Auth (OIDC) to authenticate.

Icon	Meaning
🔓	Public (no auth required)
🔒	Requires authentication
👑	Requires admin access

Auth Endpoints

GET /api/auth/user 🔒

Get the current authenticated user.

Response:

```
{
  "id": "uuid",
  "email": "user@example.com",
  "firstName": "John",
  "lastName": "Doe",
  "profileImageUrl": "https://...",
  "createdAt": "2024-01-01T00:00:00.000Z"
}
```

Clothing Items

GET /api/clothing-items 🔓

Browse the clothing catalog.

Query Parameters:

Parameter	Type	Description
category	string	Filter by category
limit	number	Max items to return

Response:

```
[
  {
    "id": "uuid",
    "name": "Classic T-Shirt",
```

```
  "brand": "Fashion Co",
  "category": "tops",
  "description": "A comfortable cotton t-shirt",
  "imageUrl": "https://...",
  "price": "29.99",
  "sizesAvailable": ["S", "M", "L", "XL"],
  "isActive": true
 }
]
```


GET /api/clothing-items/:id 🔓

Get a single clothing item by ID.

Response: Single clothing item object

User Closet

GET /api/closet 🔒

Get the authenticated user's saved items.

Response:


```
[
 {
  "id": "entry-uuid",
  "userId": "user-uuid",
  "clothingItemId": "item-uuid",
  "addedAt": "2024-01-01T00:00:00.000Z",
  "clothingItem": { /* full item object */ }
 }
]
```

POST /api/closet 🔒

Add an item to the user's closet.

Request Body:

```
{
  "clothingItemId": "item-uuid"
}
```

Response: 201 Created with entry object

DELETE /api/closet/:clothingItemId 🔒

Remove an item from the user's closet.

Response: 204 No Content

Saved Outfits

GET /api/outfits 🔒

Get the user's saved outfit combinations.

Response:

```
[
  {
    "id": "uuid",
    "userId": "user-uuid",
    "name": "Summer Look",
    "itemIds": ["item-1", "item-2", "item-3"],
    "previewImageUrl": "https://...",
    "createdAt": "2024-01-01T00:00:00.000Z"
  }
]
```

POST /api/outfits 🔒

Create a new saved outfit.

Request Body:

```
{
  "name": "Summer Look",
  "itemIds": ["item-1", "item-2"],
  "previewImageUrl": "https://..."
}
```

Response: 201 Created with outfit object

DELETE /api/outfits/:id 🔒

Delete a saved outfit.

Response: 204 No Content

Try-On Sessions

GET /api/try-on-sessions 🔒

Get the user's try-on history.

Query Parameters:

Parameter	Type	Description
limit	number	Max sessions to return

Response:

```
[
  {
    "id": "uuid",
    "userId": "user-uuid",
    "userPhotoUrl": "data:image/...",
    "bodyLandmarks": [/* 33 landmarks */],
```

```
    "results": {
      "clothingItemId": "item-uuid",
      "resultImageUrl": "data:image/..."
    },
    "shareToken": "abc123",
    "createdAt": "2024-01-01T00:00:00.000Z"
  }
]
```


POST /api/try-on-sessions 🔓

Save a new try-on session.

Request Body:


```
{
  "userPhotoUrl": "data:image/...",
  "bodyLandmarks": [/* landmarks array */],
  "results": {/* result data */}
}
```


Response: 201 Created with session object (includes generated shareToken)

Shared Results (Public)

GET /api/shared/:token 🔓

Get a shared try-on result by share token.

Response: Try-on session object or 404 Not Found

GET /api/shared/:token/votes 🔓

Get vote summary for a shared result.

Response:


```
{
```

"love": 5,

"like": 3,

"meh": 1,

"dislike": 0

}

POST /api/shared/:token/votes 🔓

Cast a vote on a shared result.

Request Body:

{

"voteType": "love",

"voterName": "Jane"

}

Valid vote types: love, like, meh, dislike

UI Label	API Value
"Love it!"	love
"Nice"	like
"Meh"	meh
"Not for me"	dislike

Response: 201 Created with vote object

GET /api/shared/:token/comments 🔓

Get comments for a shared result.

Response:

[

```
  {
    "id": "uuid",
    "authorName": "Jane",
    "content": "Love this outfit!",
    "createdAt": "2024-01-01T00:00:00.000Z"
  }
]
```


POST /api/shared/:token/comments 🔓

Add a comment to a shared result.

Request Body:


```
{
  "authorName": "Jane",
  "content": "This looks great on you!"
}
```


Response: 201 Created with comment object

Models

GET /api/models 🔓

Get available pre-photographed models.

Query Parameters:

Parameter	Type	Description
bodyType	string	Filter by body type

Response:


```
[
  {
```

```
  "id": "uuid",
  "name": "Model A",
  "photoUrl": "https://...",
  "height": "5'6\"",
  "bodyType": "average",
  "skinTone": "medium",
  "bodyLandmarks": [/* pre-detected landmarks */]
 }
]
```

Admin Endpoints 👑

All admin endpoints require authentication AND admin access (email in ADMIN_EMAILS).

GET /api/admin/check 👑

Check if current user is an admin.

Response: { "isAdmin": true } or 403 Forbidden

GET /api/admin/clothing-items 👑

List all clothing items with pagination.

Query Parameters:

Parameter	Type	Description
category	string	Filter by category
search	string	Search in name/brand
limit	number	Items per page
offset	number	Pagination offset

Parameter	Type	Description
includeInactive	boolean	Include soft-deleted items

Response:

```
{
  "items": [/* clothing items */],
  "total": 150
}
```

POST /api/admin/clothing-items 👑

Create a new clothing item.

Request Body: Clothing item fields (see schema)

Response: 201 Created with item object

PATCH /api/admin/clothing-items/:id 👑

Update a clothing item.

Request Body: Partial clothing item fields

Response: Updated item object

DELETE /api/admin/clothing-items/:id 👑

Soft delete a clothing item (sets isActive: false).

Response: 204 No Content

GET /api/admin/models 👑

List all models with pagination.

Query Parameters: Same as clothing items

POST /api/admin/models 👑

Create a new model.

PATCH /api/admin/models/:id 👑

Update a model.

DELETE /api/admin/models/:id 👑

Soft delete a model.

Object Storage

GET /public-objects/:filePath 🔓

Access public objects from storage.

GET /objects/:objectPath 🔒

Access private objects (with permission check).

POST /api/objects/upload 🔒

Get a signed upload URL for object storage.

Response:

```
{
  "uploadURL": "https://storage.googleapis.com/..."
}
```

Frontend Structure

Directory Layout

```
client/
├── src/
│   ├── components/
│   │   ├── ui/              # Shadcn UI components
│   │   ├── TryOnCanvas.tsx      # Main try-on rendering
│   │   ├── PhotoUploader.tsx     # Photo upload component
│   │   ├── SizeRecommendation.tsx # Size suggestion UI
│   │   ├── ShareModal.tsx       # Sharing dialog
```

```
│   │   └── BottomNav.tsx        # Mobile navigation
│   ├── hooks/
│   │   ├── use-auth.ts          # Authentication hook
│   │   ├── usePoseDetection.ts  # MediaPipe integration
│   │   └── use-toast.ts         # Toast notifications
│   ├── lib/
│   │   ├── queryClient.ts       # TanStack Query setup
│   │   └── utils.ts             # Utility functions
│   ├── pages/
│   │   ├── Landing.tsx          # Onboarding page
│   │   ├── Home.tsx             # Dashboard
│   │   ├── Upload.tsx           # Photo upload
│   │   ├── Catalog.tsx          # Clothing browser
│   │   ├── Studio.tsx           # Try-on workspace
│   │   ├── Closet.tsx           # Saved items
│   │   ├── Models.tsx           # Model gallery
│   │   ├── Profile.tsx          # User account
│   │   ├── SharedResult.tsx     # Public sharing page
│   │   └── Admin.tsx            # Admin dashboard
│   ├── store/
│   │   └── appStore.ts          # Zustand global state
│   ├── App.tsx                  # Routes and providers
│   └── main.tsx                 # Entry point
```

Key Components

TryOnCanvas

The core rendering component for virtual try-on.

- Renders user photo on HTML5 Canvas
- Overlays clothing images at body landmark positions
- Supports zoom, pan, and rotation
- Generates result images as data URLs

AppStore (Zustand)

Global state management for:

- Current user photo and landmarks
- Selected clothing items (multi-layer support)
- Try-on results
- Processing state

Query Client

Pre-configured TanStack Query client with:

- Default fetcher for API calls
- apiRequest helper for mutations
- Cache invalidation utilities

Computer Vision Integration

MediaPipe Pose

TryFit uses MediaPipe Pose for body landmark detection. This runs **entirely in the browser** for privacy.

33 Body Landmarks

0: nose 1: left_eye_inner 2: left_eye

3: left_eye_outer 4: right_eye_inner 5: right_eye

6: right_eye_outer 7: left_ear 8: right_ear

9: mouth_left 10: mouth_right 11: left_shoulder

12: right_shoulder 13: left_elbow 14: right_elbow

15: left_wrist 16: right_wrist 17: left_pinky

18: right_pinky 19: left_index 20: right_index

21: left_thumb 22: right_thumb 23: left_hip

24: right_hip 25: left_knee 26: right_knee

27: left_ankle 28: right_ankle 29: left_heel

30: right_heel 31: left_foot_index 32: right_foot_index

Landmark Data Structure

```
interface BodyLandmark {
  x: number;      // Normalized X (0-1)
  y: number;      // Normalized Y (0-1)
  z: number;      // Depth (relative)
  visibility: number; // Confidence (0-1)
}
```

Measurement Calculation

Body measurements are derived from landmarks:

- **Shoulder Width**: Distance between landmarks 11 and 12
- **Torso Height**: Distance from shoulder midpoint to hip midpoint
- **Hip Width**: Distance between landmarks 23 and 24

These measurements are compared to clothing size charts for recommendations.

Authentication Flow

OpenID Connect with Replit Auth

```
┌──────────┐   ┌──────────────┐   ┌──────────────────┐
│ User     │──>│ TryFit App   │──>│ Replit Auth      │
│          │   │              │   │ (OIDC Provider)  │
└──────────┘   └──────────────┘   └──────────────────┘
```

```
│          │              │
│ 1. Click Login │          │
│──────────────────>│          │
│          │ 2. Redirect to   │
│          │   Replit login   │
│          │──────────────────────>│
│          │              │
│ 3. Enter Replit │          │
│   credentials │          │
│──────────────────────────────────────-->│
│          │              │
│          │ 4. Callback with  │
│          │   auth code     │
│          │<──────────────────────│
│          │              │
│          │ 5. Exchange for   │
│          │   tokens       │
│          │──────────────────────>│
│          │              │
│          │ 6. User info +    │
│          │   session created │
│          │<──────────────────────│
│          │              │
│ 7. Logged in,  │          │
│   redirect home│          │
│<──────────────────│          │
```

Session Management

- Sessions stored in PostgreSQL (sessions table)
- Session cookie: connect.sid
- Secure: true (HTTPS required)
- HttpOnly: true (prevents XSS)

Deployment Guide

Environment Variables

Required Variables

Variable	Description	Example
DATABASE_URL	PostgreSQL connection string	postgresql://user:pass@host:5432/d
SESSION_SECRET	Secret for session encryption	Random 32+ character string

Auto-Provided by Replit

Variable	Description
REPL_ID	Unique identifier for your Repl
REPLIT_DEPLOYMENT	Deployment identifier
ISSUER_URL	OIDC issuer (defaults to Replit)

Optional Variables

Variable	Description	Default
NODE_ENV	Environment mode	developr

Variable	Description	Default
ADMIN_EMAILS	Comma-separated admin emails	(none)
PUBLIC_OBJECT_SEARCH_PATHS	Object storage paths	(none)

Example Configuration

```
# Required
DATABASE_URL=postgresql://user:password@db.example.com:5432/tryfit
SESSION_SECRET=your-super-secret-session-key-at-least-32-chars
# Optional
NODE_ENV=production
ADMIN_EMAILS=admin@example.com,manager@example.com
```

Database Setup

1. Create Database

TryFit uses PostgreSQL. On Replit, a database is automatically provisioned.

For external PostgreSQL (e.g., Neon):

1. Create a new database
2. Copy the connection string
3. Set DATABASE_URL in Replit Secrets

2. Run Migrations

```
# Push schema to database
npm run db:push
# If you encounter conflicts, use force flag (be careful in production)
npm run db:push --force
```

This uses Drizzle Kit to sync the schema from shared/schema.ts to your PostgreSQL database.

Important: The sessions table is required for authentication. It will be created automatically when you push the schema.

3. Seed Data (Optional)

If you have a seed script:

npx tsx server/seed.ts

This populates the database with sample clothing items and models for testing.

Running Locally

Prerequisites

- Node.js 20+
- PostgreSQL database
- Replit account (for authentication)

Steps

1. **Clone the repository**

git clone <repo-url>

cd tryfit

2. **Install dependencies**

npm install

3. **Set up environment** Create a .env file:

DATABASE_URL=postgresql://...

SESSION_SECRET=dev-secret-key-for-local-development

4. **Run database migrations**

npm run db:push

5. **Start development server**

npm run dev

6. **Open in browser** Navigate to http://localhost:5000

Publishing to Replit

Deployment Options

Replit offers several deployment options:

Type	Best For	Description
Autoscale	Web apps with variable traffic	Scales up/down automatically
Reserved VM	Consistent workloads	Dedicated resources
Static	Static sites only	Most cost-effective

Recommended for TryFit: Autoscale Deployments

Deployment Steps

1. **Prepare your app**
    - Ensure all environment variables are set in Secrets
    - Test the app thoroughly in development mode
    - Verify database migrations are applied
2. **Click the Publish button**
    - In your Replit workspace, click the "Publish" icon (rocket)
    - Select "Autoscale Deployments"
3. **Configure deployment**
    - Choose your plan and resources
    - Add payment method if required
4. **Deploy**
    - Click "Publish" to deploy
    - Your app will be available at your-repl.replit.app
5. **Custom Domain (Optional)**

    - Go to deployment settings
    - Add your custom domain
    - Configure DNS as instructed

Post-Deployment Checklist

- Test authentication flow
- Verify database connectivity
- Test image uploads
- Check admin access (if applicable)
- Test sharing functionality

Admin Configuration

Granting Admin Access

1. **Set ADMIN_EMAILS** Add the environment variable in Replit Secrets:

ADMIN_EMAILS=admin@example.com,other-admin@example.com

2. **Access Admin Dashboard**
    - Log in with an admin email
    - Navigate to /admin
    - You'll see the catalog management interface

Admin Capabilities

- **Clothing Items**: Create, edit, soft-delete items
- **Models**: Add and manage pre-photographed models
- **Bulk Operations**: Edit multiple items at once
- **Inactive Items**: View and restore soft-deleted items

Production Considerations

Security

1. **HTTPS Required**
    - Session cookies are secure (HTTPS only in production)
    - Replit deployments include automatic HTTPS
    - Session cookie settings: secure: true, httpOnly: true
2. **Admin Protection**
    - Admin routes require both authentication AND admin email match
    - ADMIN_EMAILS is checked server-side via isAdmin middleware
    - Never expose ADMIN_EMAILS publicly
3. **Rate Limiting (Recommended)**
    - Consider adding rate limiting for public endpoints
    - Public voting (POST /api/shared/:token/votes) is unauthenticated
    - Public comments (POST /api/shared/:token/comments) is unauthenticated
    - Implement rate limiting (e.g., express-rate-limit) to prevent spam/abuse
4. **Input Validation**
    - All inputs are validated using Zod schemas
    - Vote types restricted to: love, like, meh, dislike
    - Comment content and author names are trimmed and required

Performance

1. **Database Connection Pooling**
    - Uses Neon serverless with WebSocket connections
    - Connection pooling is handled automatically
2. **Image Optimization**
    - Consider compressing uploaded images
    - Use thumbnails for catalog views
3. **Caching**
    - TanStack Query provides client-side caching

- Consider CDN for static assets

Monitoring

1. **Error Tracking**
    - Server errors are logged to console
    - Consider adding error tracking (e.g., Sentry)
2. **Analytics**
    - Track user engagement and try-on sessions
    - Monitor popular clothing items

Backup

1. **Database Backups**
    - Replit/Neon provides automatic backups
    - Consider manual exports for critical data
2. **Code Versioning**
    - Use Git for version control
    - Replit provides automatic checkpoints